Software Architecture 2024 [2026]
Lecture 1, part 3/3

Software engineering process and the role of requirements

Wouter Joosen

Laurens Sion

Dimitri Van Landuyt

KU LEUVEN DistriNet

Outline

1. **Software engineering processes**

Initial phases of the software development life cycle

2. **Requirements engineering**

First acquaintance with ASRs – architecturally-significant requirements

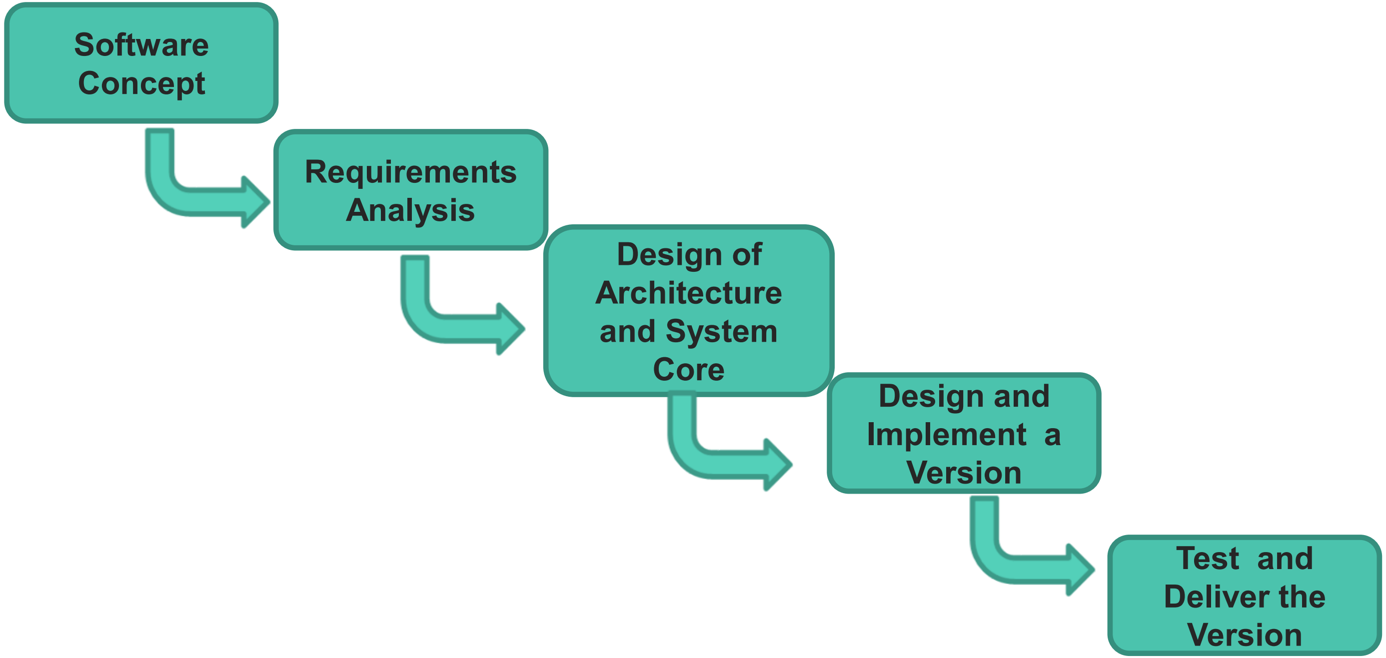
(1) Software Engineering Processes

How to get from *A* to *Z*?

Software Concept

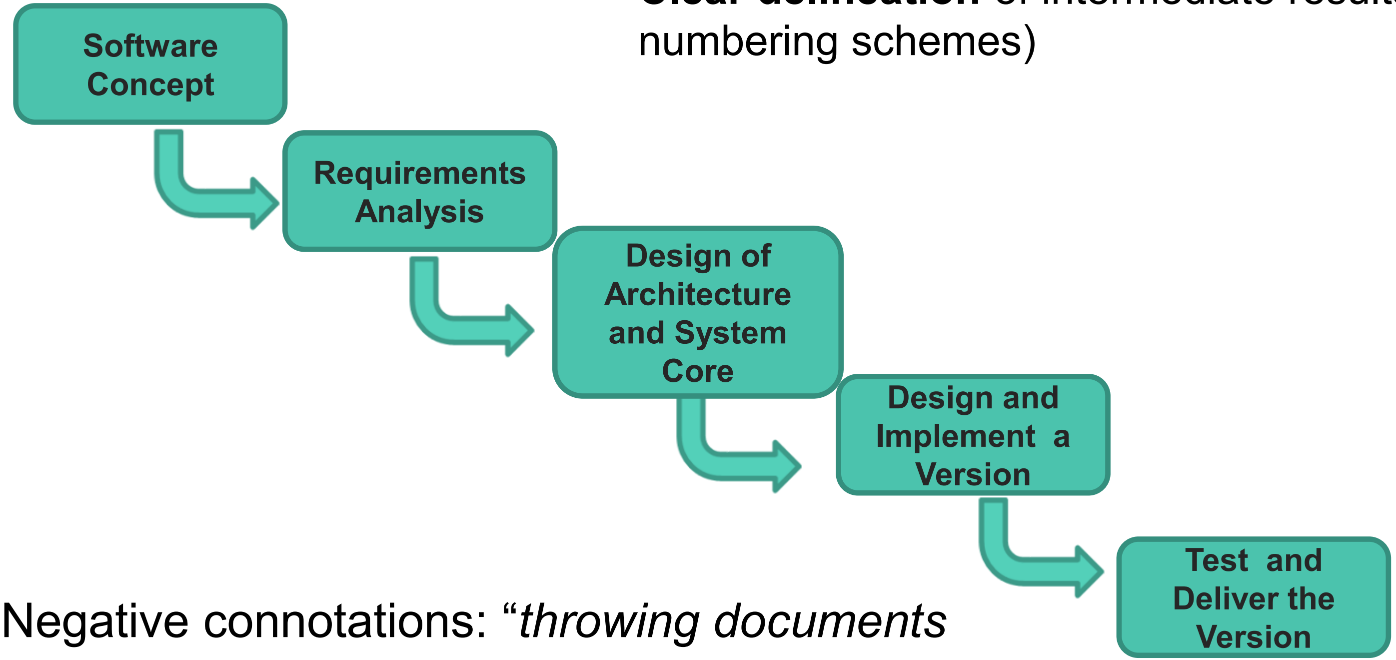
Delivery

Waterfall Model for Software Development



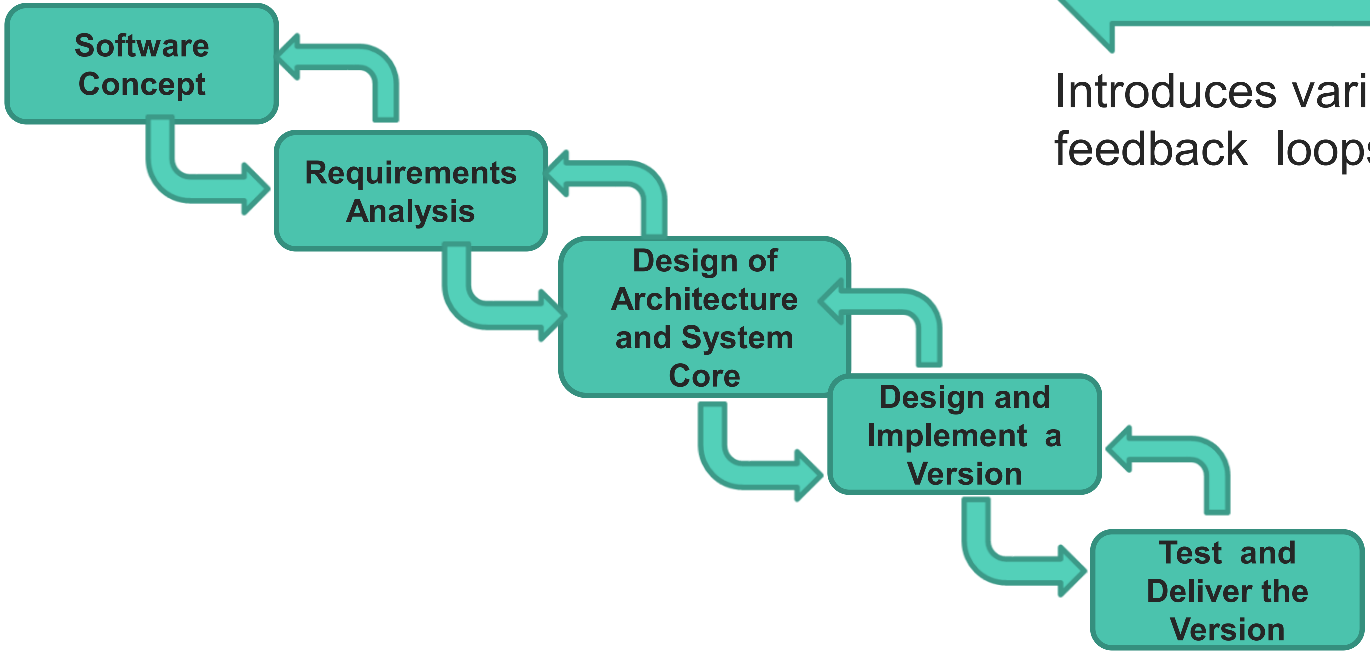
Waterfall Model for Software Development

- Reliance on complete and standalone **specifications**
- **Clear delineation** of intermediate results (e.g. version numbering schemes)



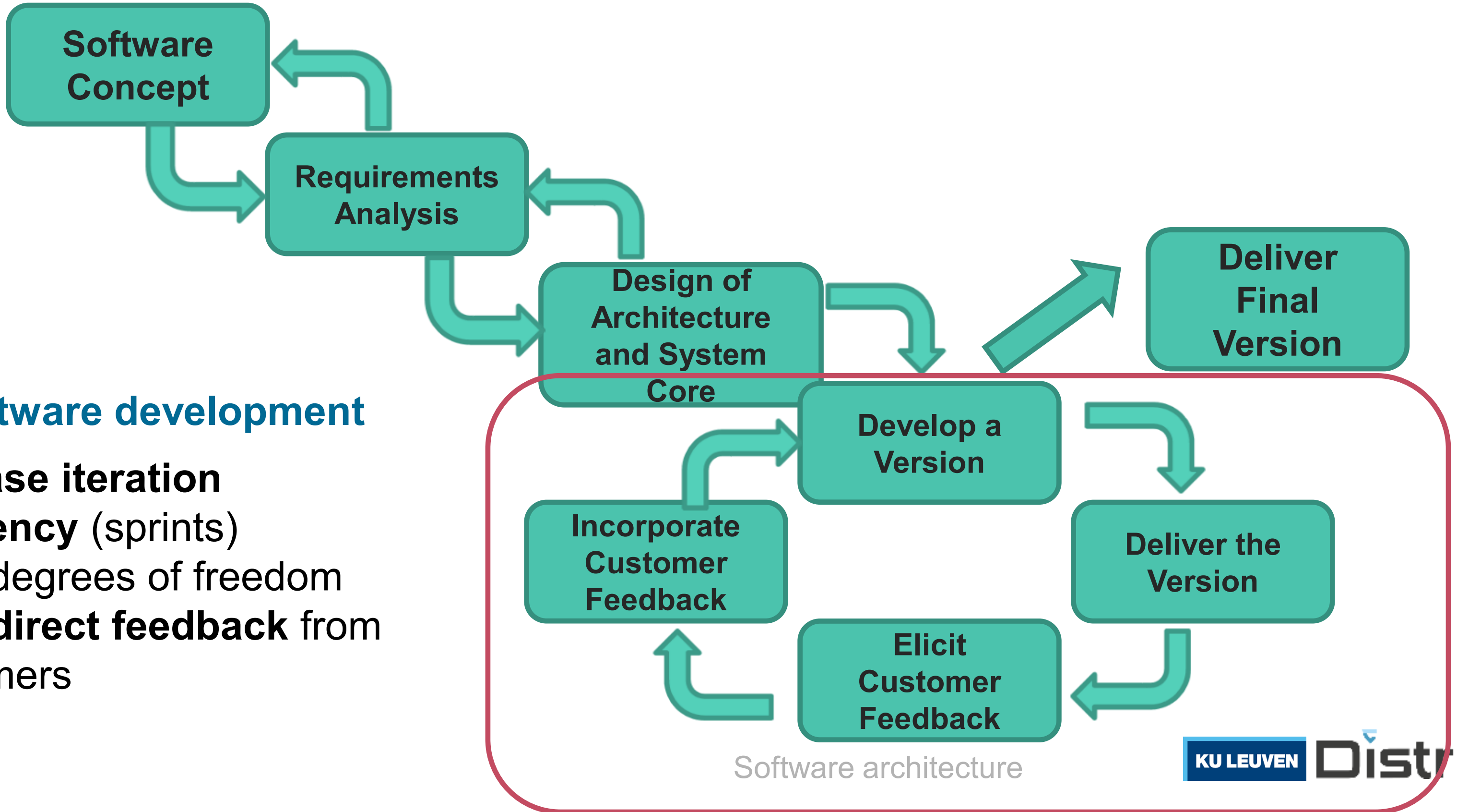
Negative connotations: “*throwing documents over the wall*”; *slow, heavyweight*

Iterative Software Development (at the basis of the Unified Process)



Introduces various feedback loops

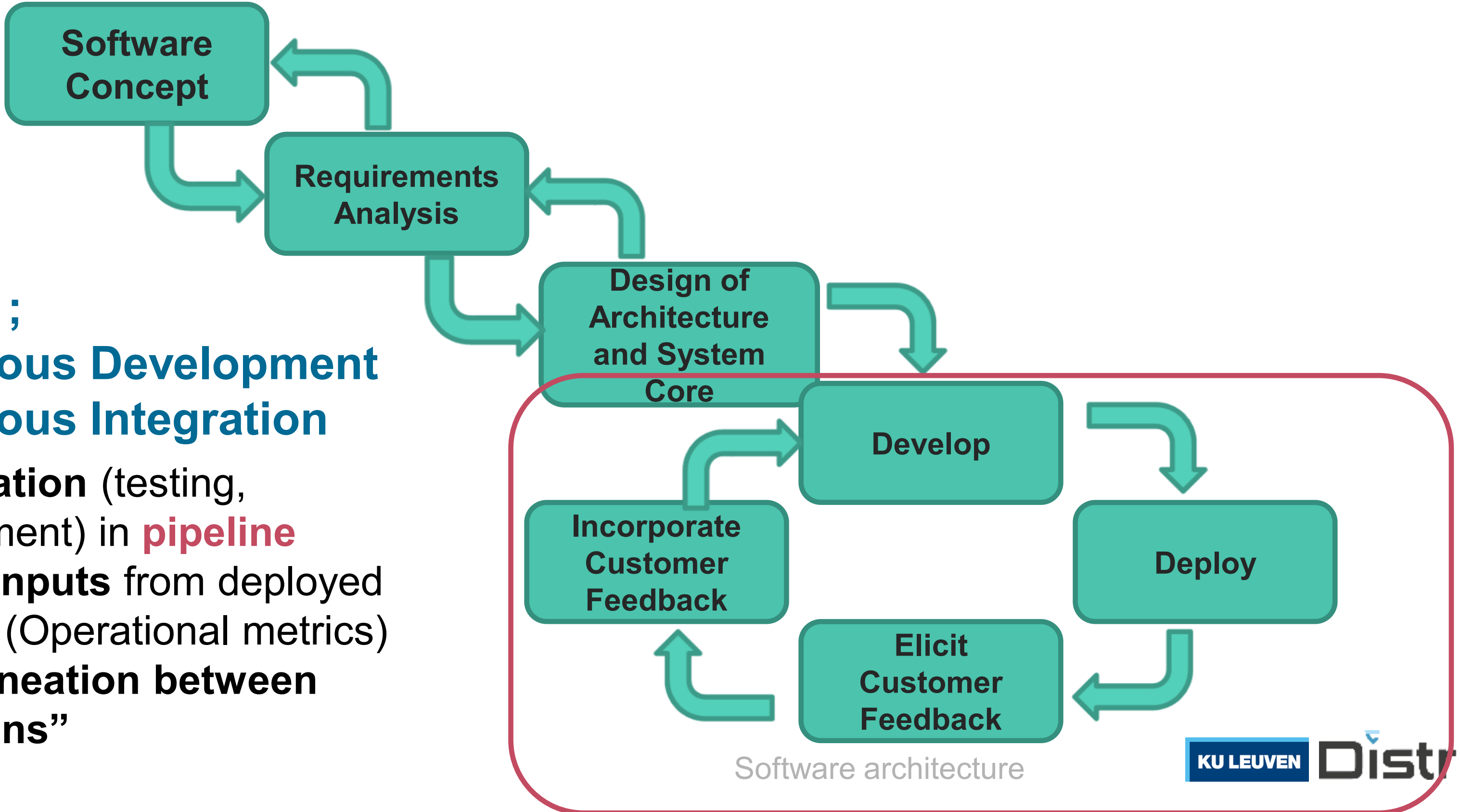
Other development processes



Agile software development

- **Increase iteration frequency** (sprints)
- More degrees of freedom
- More **direct feedback** from customers

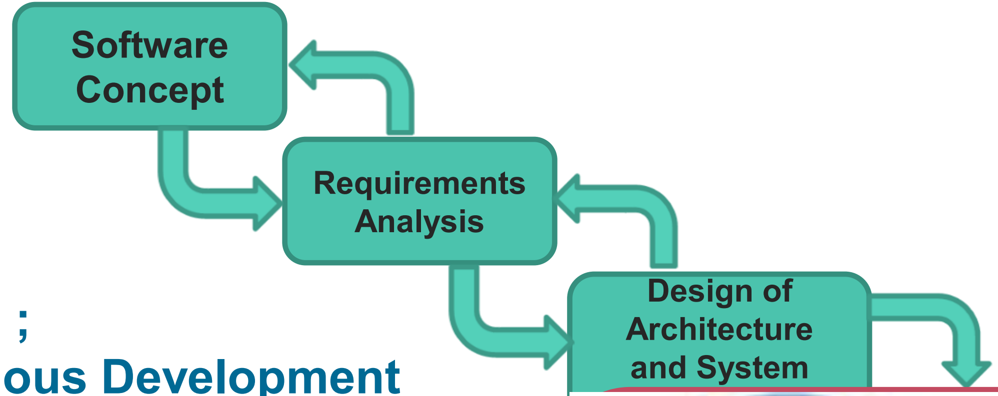
Other development processes



DevOps ; Continuous Development Continuous Integration

- **Automation** (testing, deployment) in **pipeline**
- **Direct inputs** from deployed version (Operational metrics)
- No **delineation between "versions"**

Other development processes



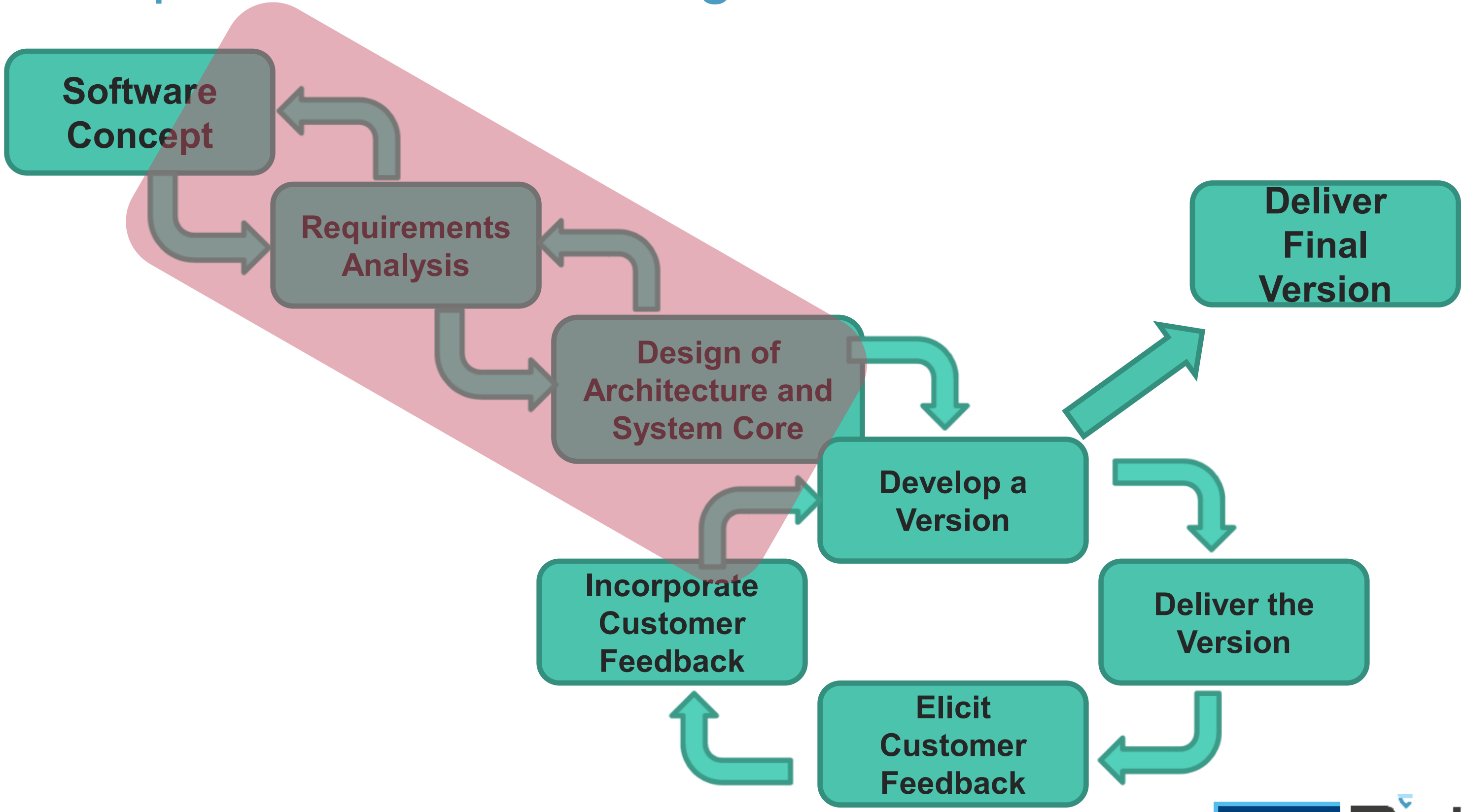
DevOps ;
Continuous Development
Continuous integration

- **Automation** (testing, deployment) in **pipeline**
- **Direct inputs** from deployed version (Operational metrics)
- No **delineation between "versions"**

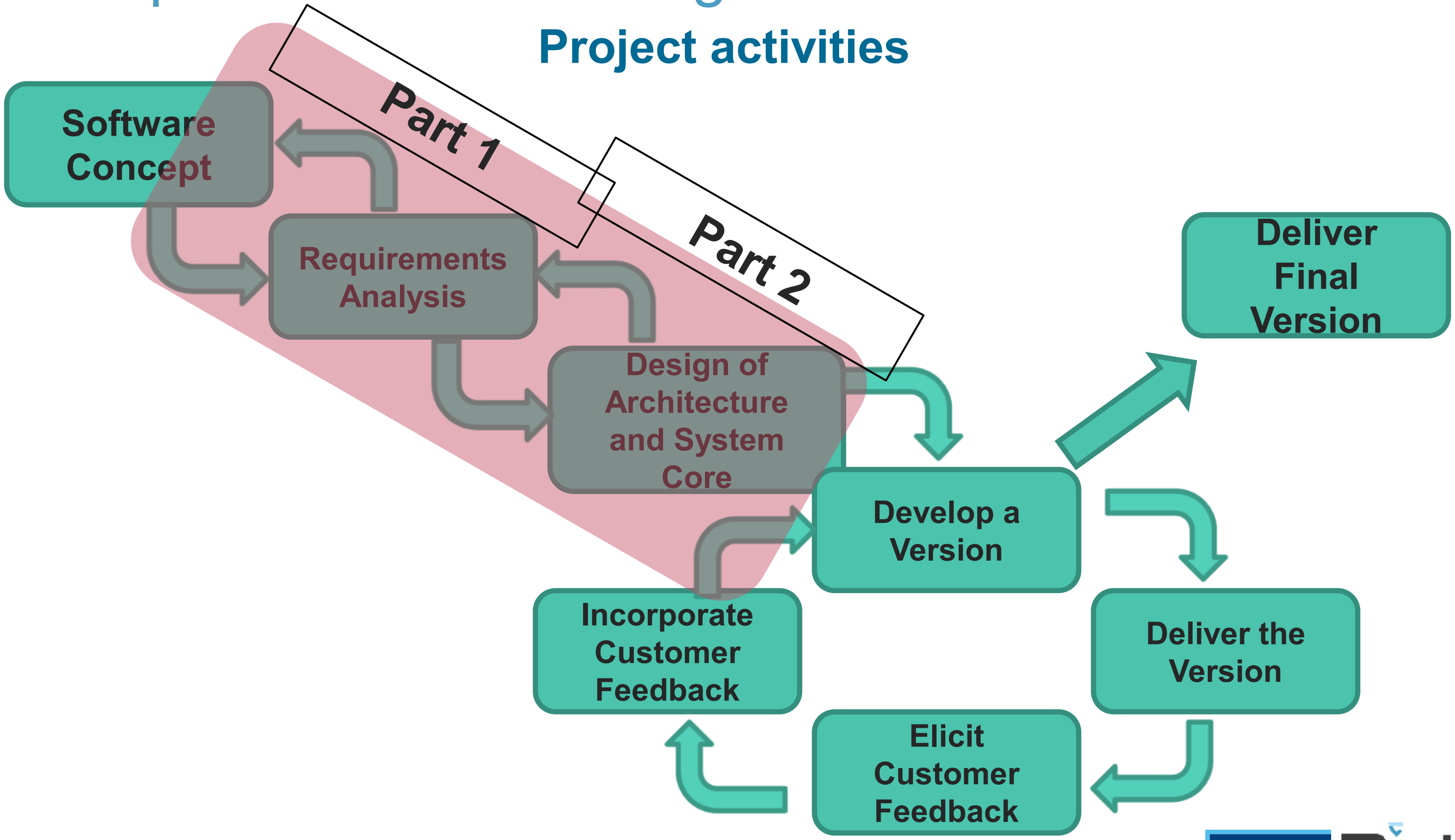
Focus in this course is on the **initial stages of development**

Regardless of your implementation process

Software architecture is the bridge between requirements and design



Software architecture is the bridge between requirements and design

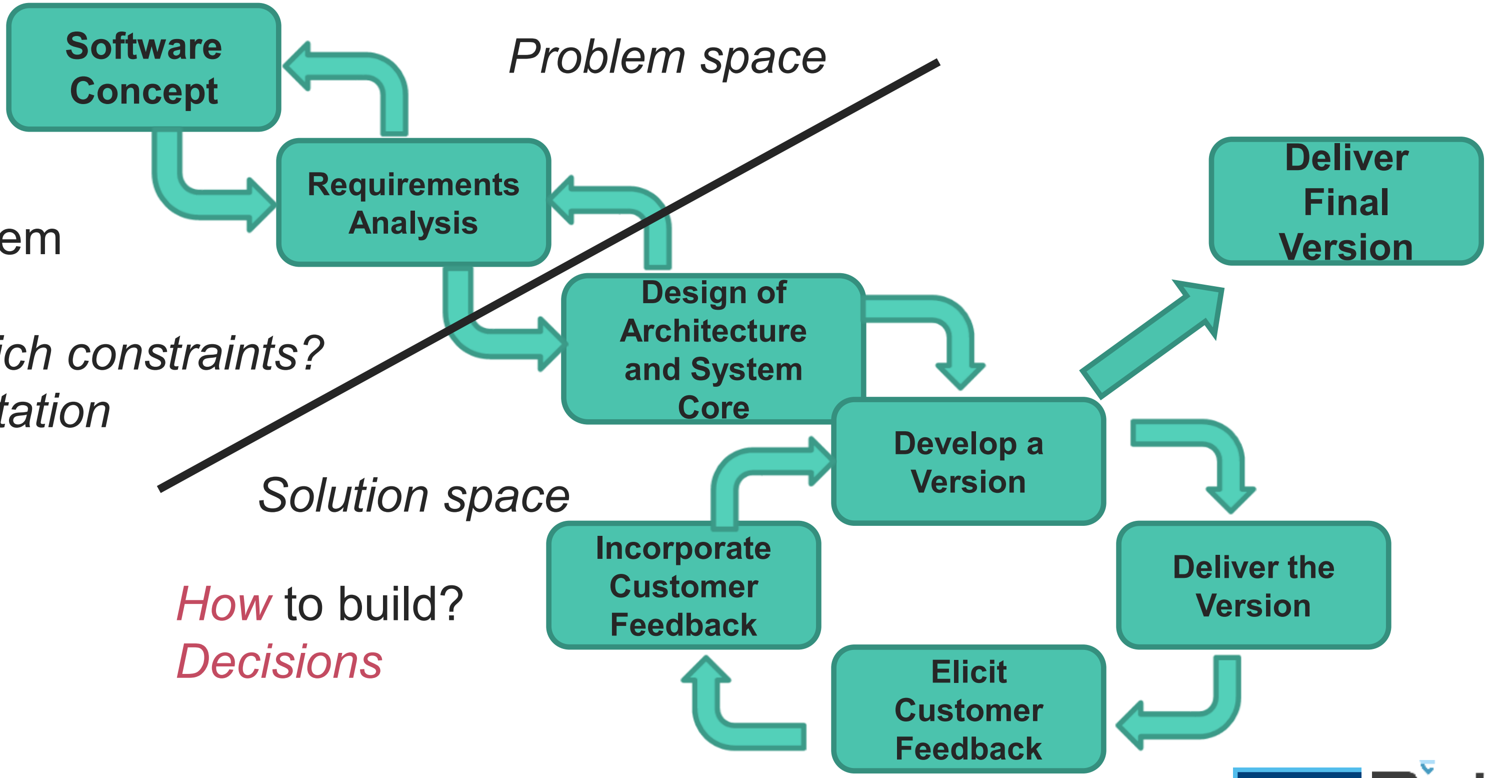


Software Architecture is the “bridge” between requirements and design

between

Problem space and **Solution space**

Architecture is bridge between requirements and design

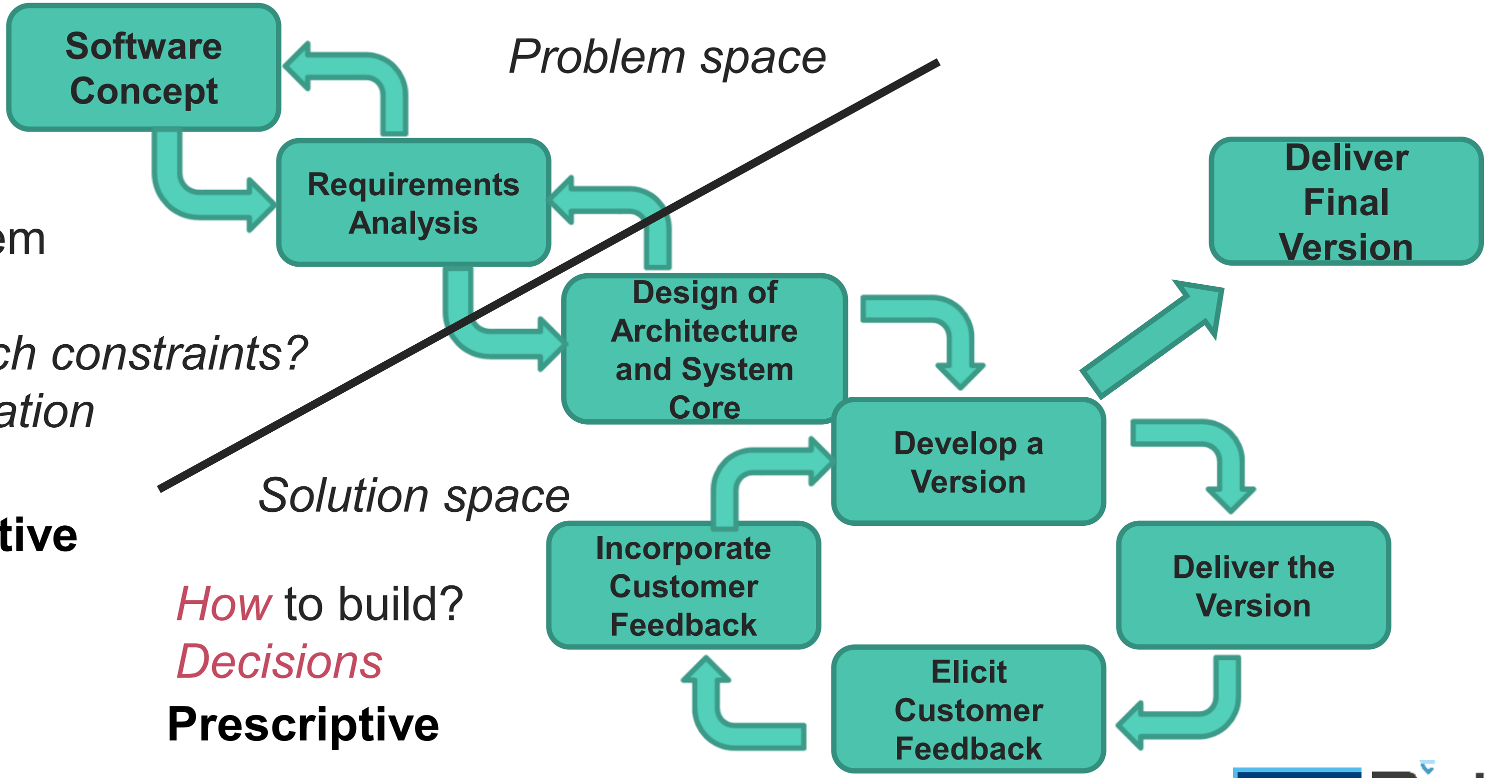


What system to build?
Under which constraints?
Representation of reality

How to build?
Decisions

KU LEUVEN DistriNet

Architecture is bridge between requirements and design



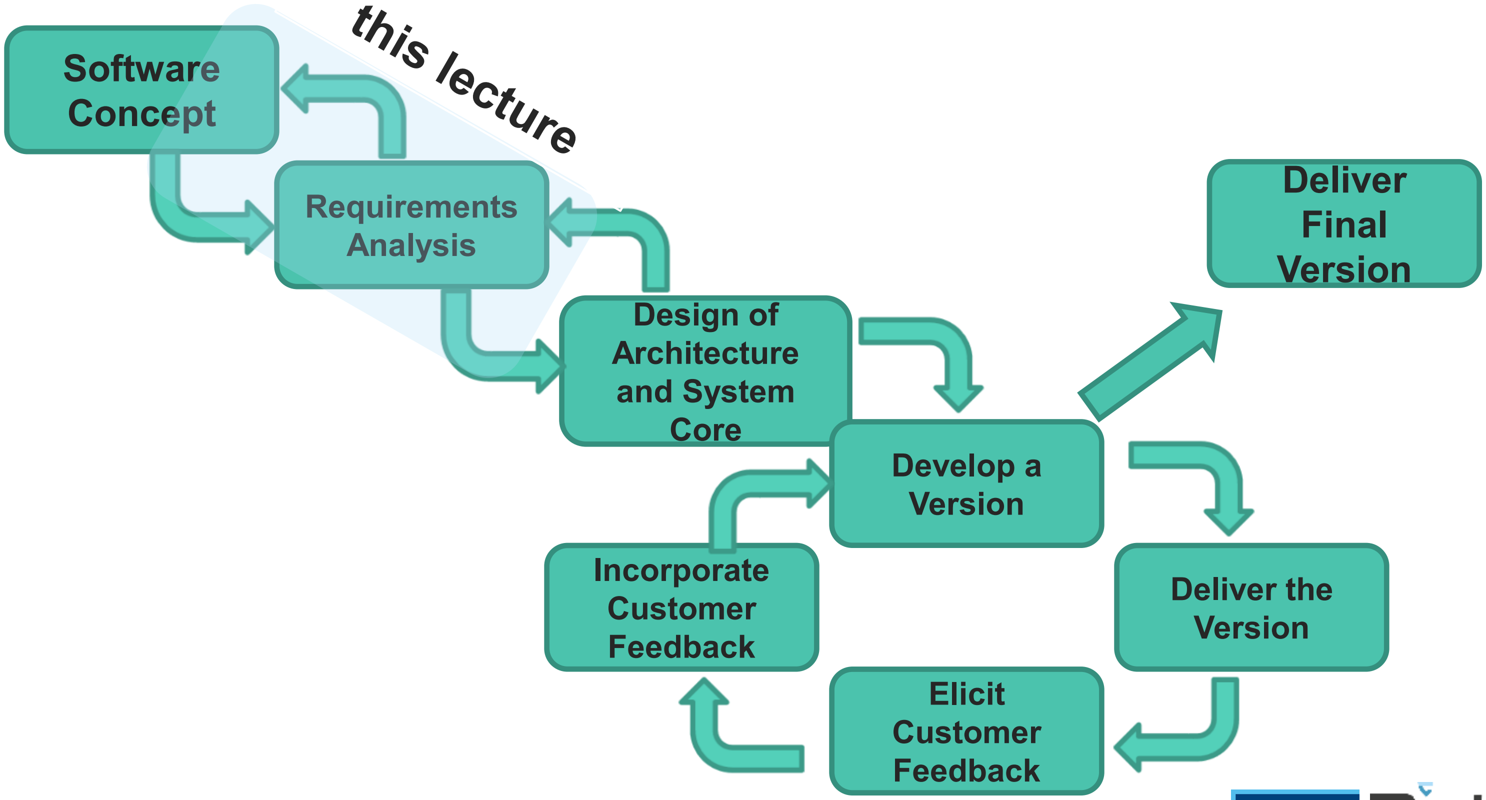
What system to build?
Under which constraints?
Representation of reality

Descriptive

How to build?
Decisions

Prescriptive

Architecture is bridge between requirements and design



(2) Requirements Engineering

THE PROJECT CONSTRUCTION CYCLE – THE TREE SWING

HOW THE CLIENT DESCRIBED IT

HOW THE ARCHITECT ENVISIONED IT

HOW THE ENGINEER DESIGNED IT

WHAT THE BUDGET ALLOWED

HOW THE LIABILITY INSURANCE AGENT DESCRIBED IT

HOW THE ESTIMATOR BID IT

HOW THE MANUFACTURER MADE IT

WHAT THE BUILDING INSPECTOR EXPECTED

HOW THE CONTRACTOR INSTALLED IT

WHAT THE CUSTOMER REALLY WANTED

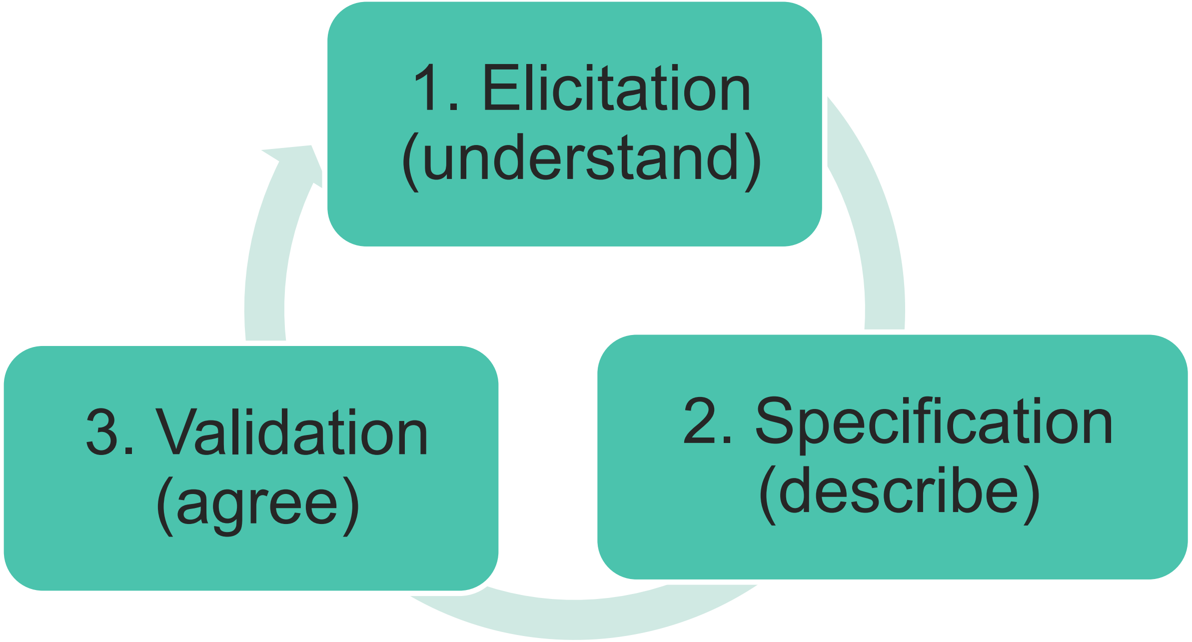
HOW THE PROJECT WAS DOCUMENTED

HOW THE CUSTOMER WAS BILLED

Definition

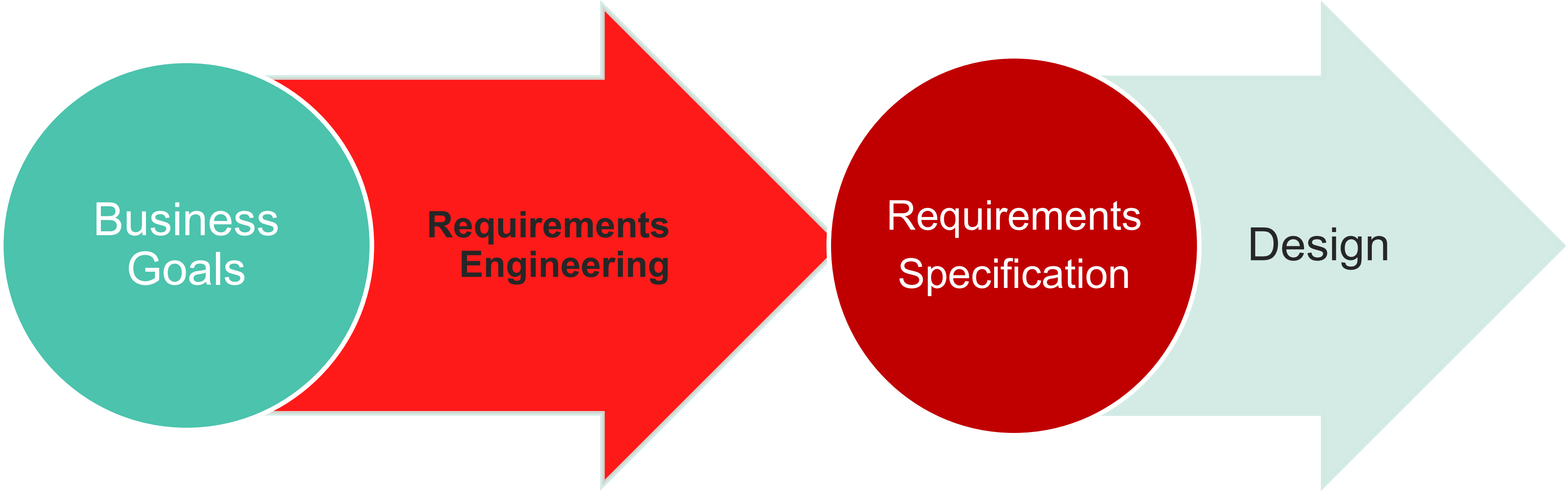
- Requirements engineering is the branch of software engineering concerned with the characterization and analysis of
  - the **real-world goals** for functions of,
  - **constraints on** software systems (“domain analysis”/ “context analysis”)
- It is also concerned with the relationship of these factors
  - to **precise specifications** of software behavior and
  - to their **evolution over time** and across software families

Requirements Engineering Process



Requirements Engineering

- First step in finding a solution



Functional vs. Non-Functional Requirements

1. **Functional requirements**
   - Functionality (“System services”/visible behavior)
2. **Non-functional requirements** or ***quality*** *requirements*
   - Constraints under which the system must operate
   - **Quality** the system must exert
   - *ilities

In this course, we rely on **scenario-based** requirements elicitated from a **black box perspective**

Functional vs. Non-Functional Requirements

1. **Functional requirements**
   - Functionality (“System services”/visible behavior)
2. **Non-functional requirements** or ***quality*** *requirements*
   - Constraints under which the system must operate
   - **Quality** the system must exert
   - *ilities

USE CASES

In this course, we rely on **scenario-based** requirements elicitated from a **black box perspective**

Functional vs. Non-Functional Requirements

1. **Functional requirements**
   - Functionality ("System services"/visible behavior)

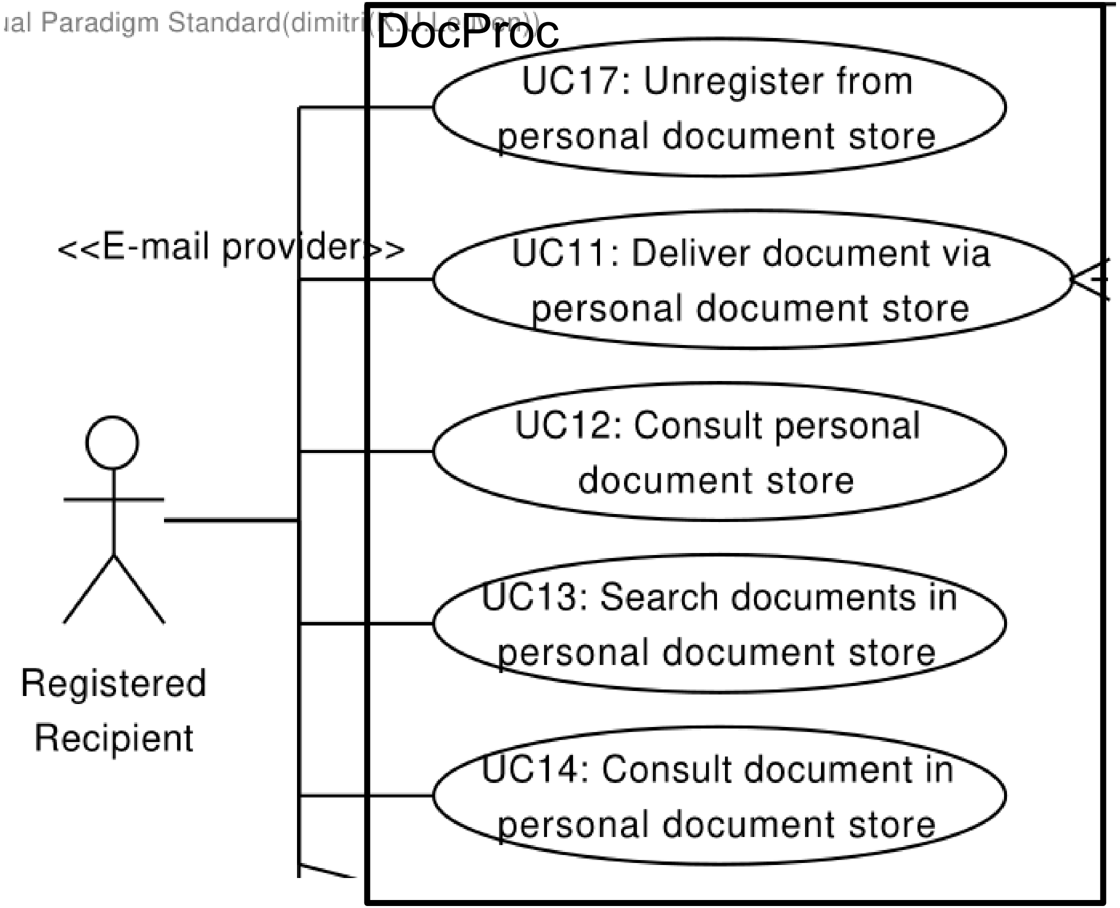
USE CASES

2. **Non-functional requirements** or ***quality*** *requirements*
   - Constraints under which the system must operate
   - **Quality** the system must exert
   - *ilities

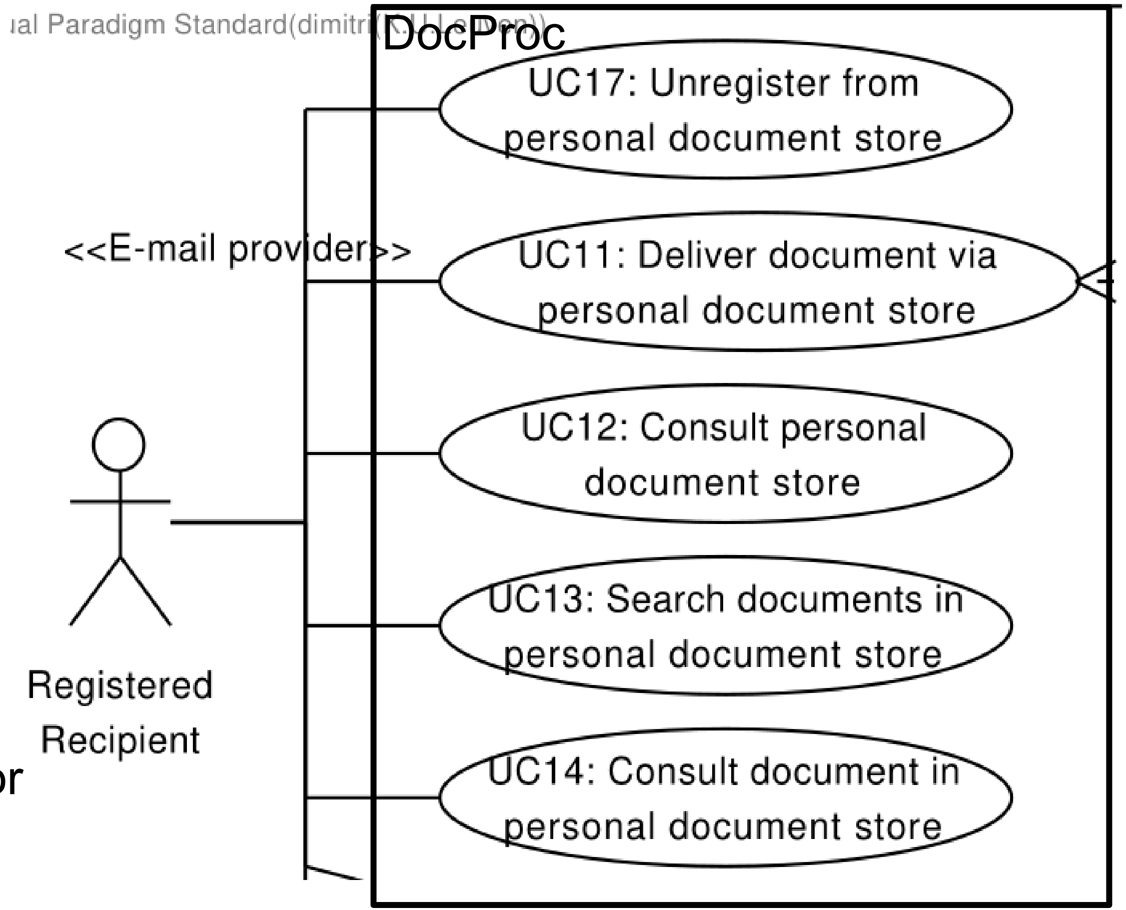
QUALITY ATTRIBUTE SCENARIOS

In this course, we rely on **scenario-based** requirements elicitated from a **black box perspective**

Use case model



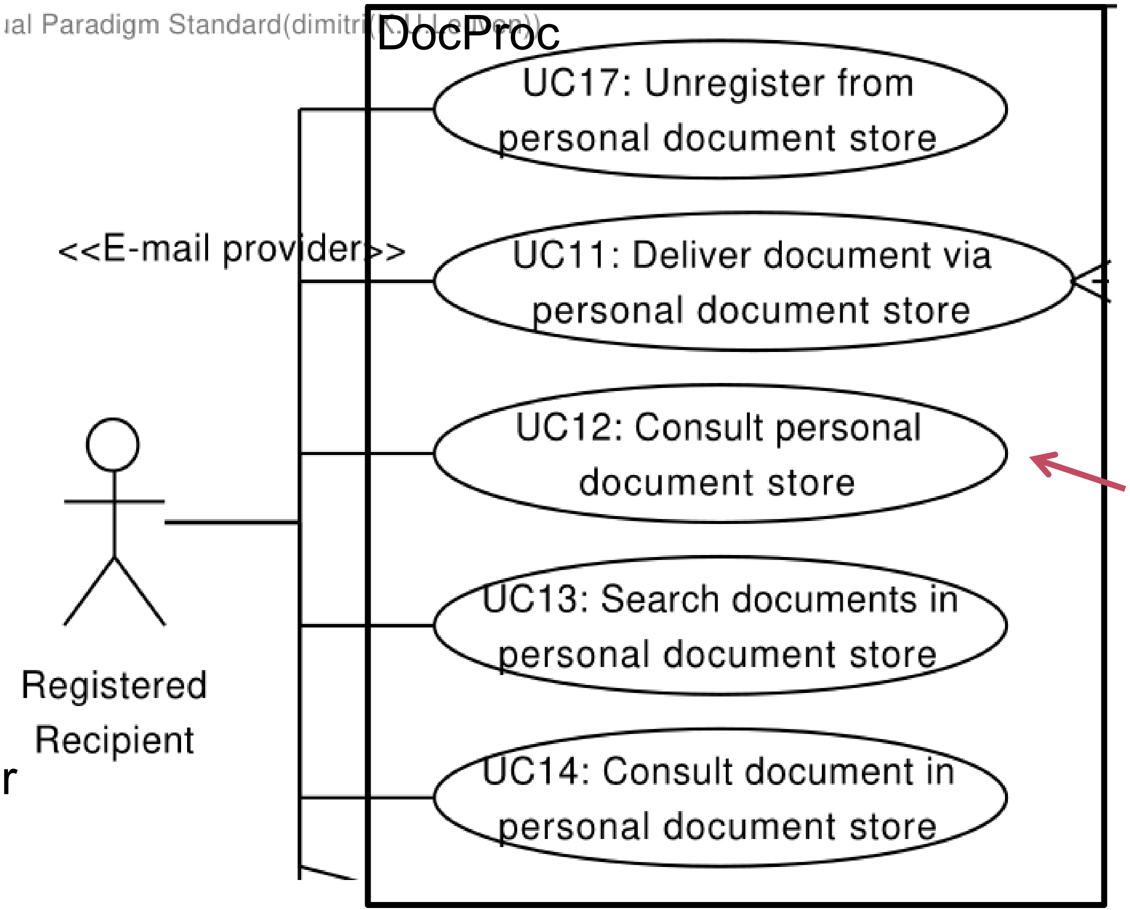
Use case model



"**Actor**" = user or external system

Functional requirements

Use case model

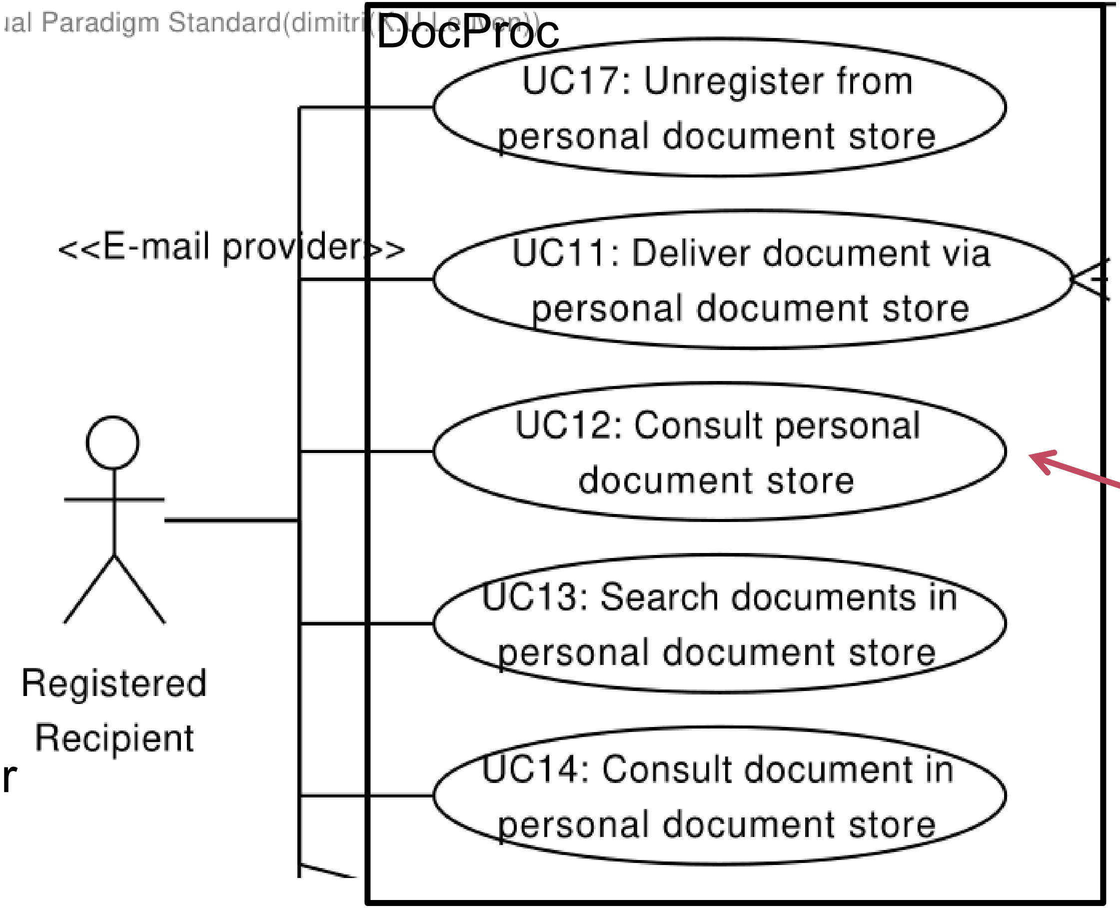


"**Actor**" = user or external system

"**Use case**" = an interaction scenario, prescriptive in how the system should respond to impulses (requests/triggers/events)

Functional requirements

Use case model



System boundary

"Use case" = an interaction scenario, prescriptive in how the system should respond to impulses (requests/triggers/events)

"Actor" = user or external system

Functional requirements

Use case: consult personal document store

1. The **Registered Recipient** indicates that he or she wants to consult his or her personal document store.
2. The **System** looks up all documents sent to the Registered Recipient.
3. The **System** provides an overview of all the documents in the personal document store of the Registered Recipient, e.g., as a table or a list, together with the possibility to consult each individual document (cf. UC14 : Consult document in personal document store) and the possibility to download each such document as a PDF.

Alternative scenarios:

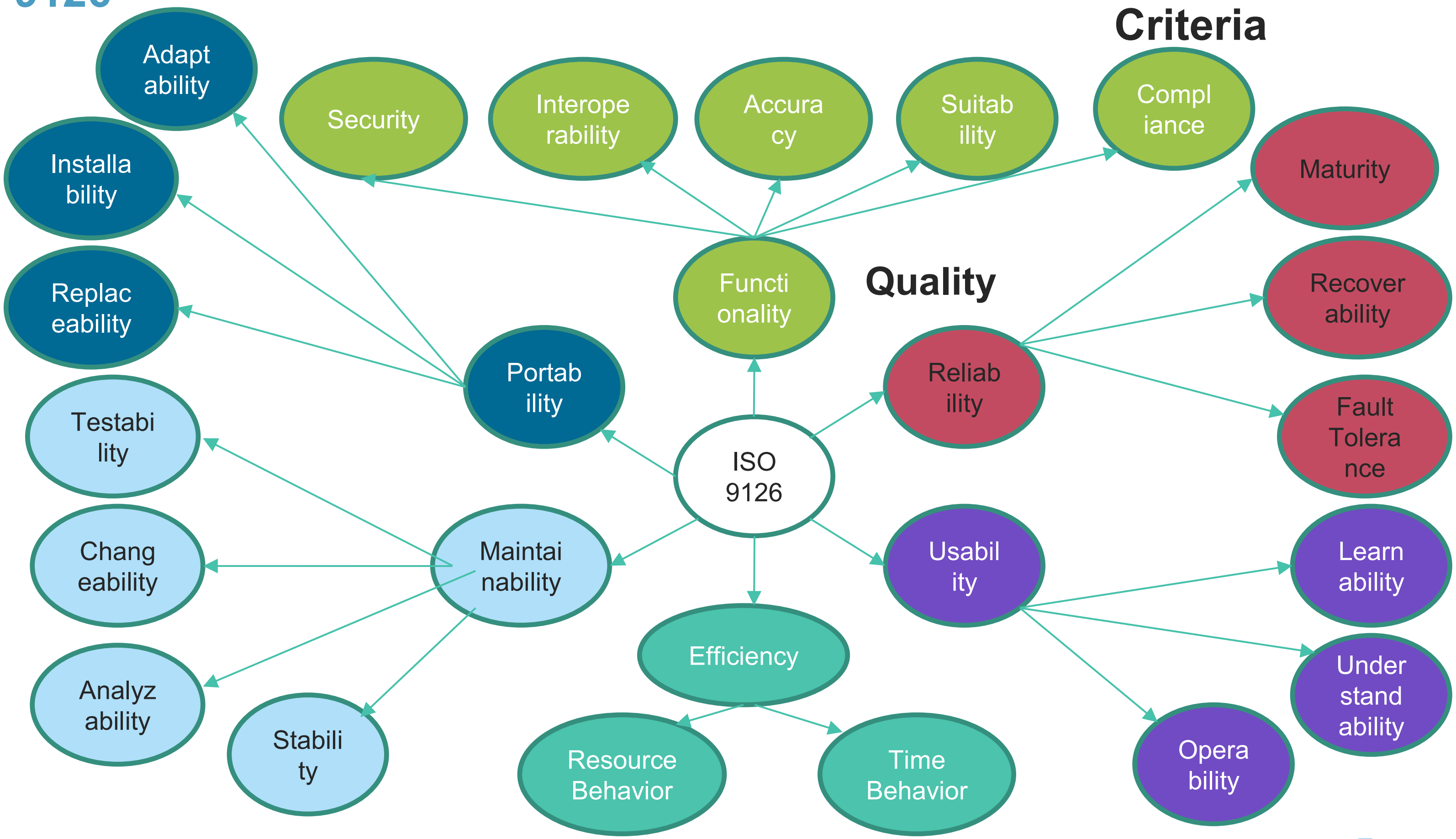
3a. If the Registered Recipient has not received any documents yet, the System indicates this to the Registered Recipient. The use case ends

Functional requirements **scenario-based** **black-box perspective**

Software Quality Models

- **Lots of ambiguity**: e.g. ***performance*** *of an AI system (precision/recall)* vs ***performance*** *of a system (throughput/latency)*?
- Aim for a **level of concretization**:
  - allow verifying that quality is achieved: **measures**
- Quality Model
  - Typically tree structures
  - High level goals are refined in low level criteria
  - Metrics are attached to criteria
- Several models: ISO, McCall, Boehm, …

ISO 9126


Criteria
Quality
ISO 9126
Functi onality
Security
Interope rability
Accura cy
Suitab ility
Compl iance
Reliab ility
Maturity
Recover ability
Fault Tolera nce
Usabil ity
Learn ability
Under stand ability
Opera bility
Efficiency
Resource Behavior
Time Behavior
Maintai nability
Testabi lity
Chang eability
Analyz ability
Stabili ty
Portab ility
Adapt ability
Installa bility
Replac eability

Quality Models: McCall

McCall, J. A., Richards, P. K., Walters, G. F. Factors in Software Quality, Volumes I, II, and III. US Rome Air Development Center Reports, US Department of Commerce, USA, 1977.

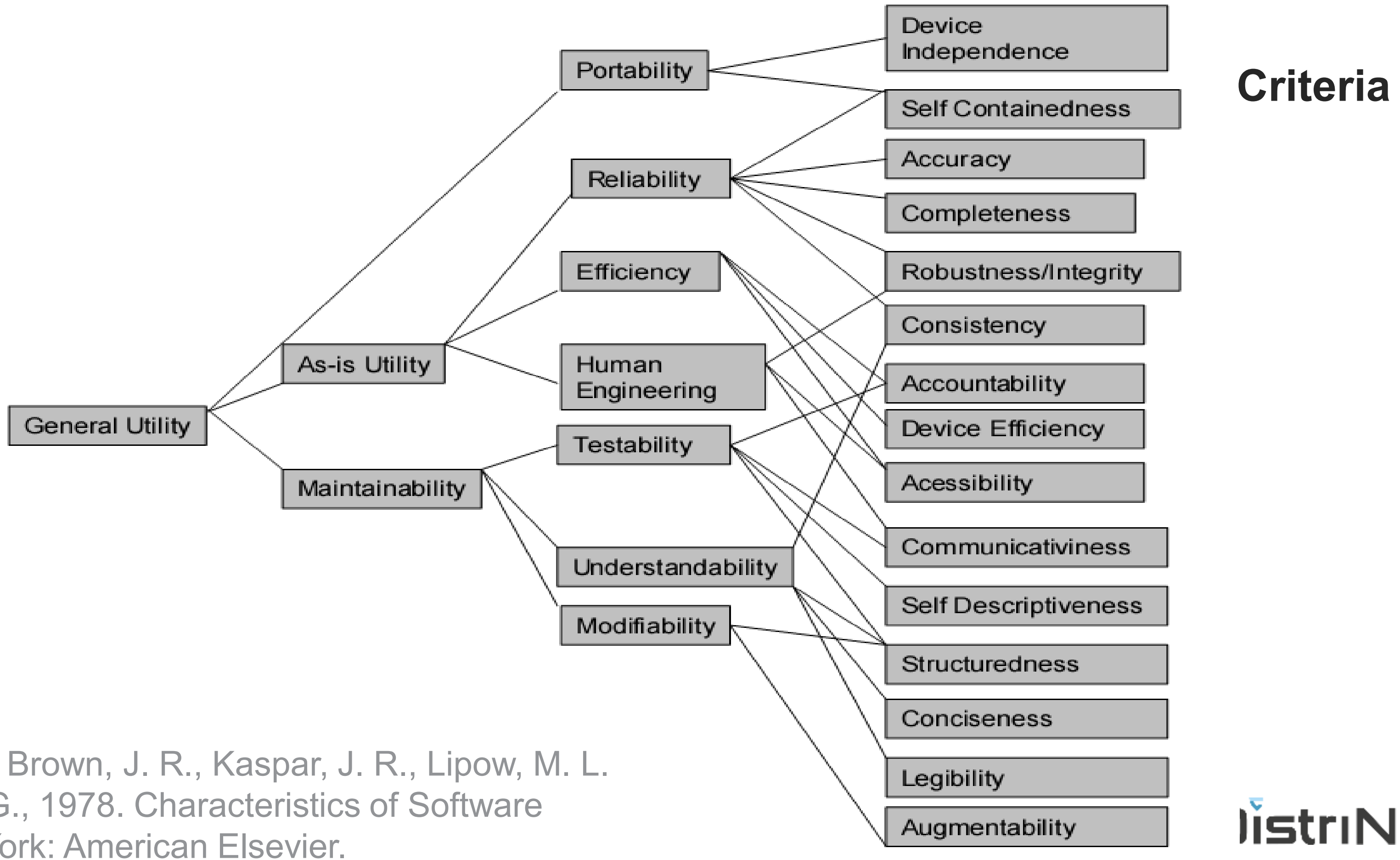
Table 2.2-2 Sources for Software Quality Factors

AUTHORS		PORTABILITY	TRANSFERABILITY	ACCEPTABILITY	COMPLETENESS	CONSISTENCY	CORRECTNESS	AVAILABILITY	RELIABILITY	ACCURACY	ROBUSTNESS	EFFICIENCY	PERFORMANCE	CONCISENESS	UNDERSTANDABILITY	CLARITY	SELF-DESCRIPTIVENESS	LEGIBILITY	MAINTAINABILITY	STABILITY
Boehm	BOEHB76	●			●	●				●		●		●	●		●	●	●	
Casey	CASEJ74								●											
Culpepper	CULPL75	●																		
Dennis	DENNJ73						●		●				●							
Edwards	EDWAN75																			
Goodenough	GOODJ75		●												●					
Goos	DENNJ70	●																		
Hague	HAGUS76	●																		
JLC/SRWG	FIND75								●											
Kernighan	KERNB74										●	●								
Kosy	KOSYD74	●		●					●			●							●	
Ledgard	KOSAS74								●							●				
Lieglein	LIEBE72		●																●	
Light	LIGHW76			●					●											
Liskov	LISKB73							●												
Marshall	MARSS70		●																	
Mealy	MEALG68		●																	
Myers	MYERG75,76								●			●							●	●
NSSC PATHWAY	PATH76		●				●													
Poole	DENNJ70	●																		
Richards	RICHP74											●								
Ruby	RUBYR68						●			●		●			●					
SADPR-85	SUPP73							●												
SAMSO	SAMS75			●					●											
Schonfelder	SCHOJ76									●	●									
Thayer	THAYT76								●											
USA [illegible]SRAD	USAR75	●		●			●													
[illegible]	WHIPL75								●						●					
Wulf	WULFW73	●									●		●			●				
Yourdon	YOURE75																			
[illegible]	[illegible]							●	●	●		●							●	●

AUTHORS		ADAPTABILITY	EXTENSIBILITY	MODIFIABILITY	ACCESSIBILITY	FLEXIBILITY	EXPANDABILITY	AUGMENTABILITY	INTEGRITY	SECURITY	PRIVACY	USABILITY	OPERABILITY	HUMAN FACTORS	COMMUNICATIVENESS	STRUCTUREDNESS	MODULARITY	UNIFORMITY	GENERALITY	REUSABILITY
Boehm	BOEHB76			●	●			●				●		●	●		●			
Casey	CASEJ74																			
Culpepper	CULPL75					●						●					●			
Dennis	DENNJ73																			
Edwards	EDWAN75																			
Goodenough	GOODJ75			●														●		●
Goos	DENNJ70															●				
Hague	HAGUS76																			
JLC/SRWG	FIND75																			
Kernighan	KERNB74															●				
Kosy	KOSYD74	●				●			●	●			●						●	
Ledgard	KOSAS74					●														
Lieglein	LIEBE72																			
Light	LIGHW76					●			●											
Liskov	LISKB73																			
Marshall	MARSS70																			
Mealy	MEALG68																●			
Myers	MYERG75,76	●		●						●		●		●					●	
NSSC PATHWAY	PATH76			●																
Poole	DENNJ70	●																		
Richards	RICHP74					●														
Ruby	RUBYR68			●								●					●			
SADPR-85	SUPP73		●				●			●		●								
SAMSO	SAMS75	●							●											
Schonfelder	SCHOJ76																			
Thayer	THAYT76																			
USA [illegible]SRAD	USAR75	●							●	●	●									
[illegible]	WHIPL75			●																●
Wulf	WULFW73			●										●						
Yourdon	YOURE75					●														
[illegible]	[illegible]					●			●	●									●	

AUTHORS		TESTABILITY	INTEROPERABILITY	CONVERTIBILITY	MANAGEABILITY	ACCOUNTABILITY	SELF-CONTAINEDNESS	EXPRESSION	VALIDITY	TIME	COST	COMPLEXITY	DOCUMENTATION	PRECISION	TOLERANCE	COMPATABILITY	REPAIRABILITY	SERVICEABILITY
Boehm	BOEHB76	●				●	●											
Casey	CASEJ74																	
Culpepper	CULPL75																	
Dennis	DENNJ73																	
Edwards	EDWAN75	●																
Goodenough	GOODJ75																	
Goos	DENNJ70																	
Hague	HAGUS76																	
JLC/SRWG	FIND75																	
Kernighan	KERNB74							●										
Kosy	KOSYD74		●															
Ledgard	KOSAS74																	
Lieglein	LIEBE72																	
Light	LIGHW76	●																
Liskov	LISKB73	●																
Marshall	MARSS70																	
Mealy	MEALG68																	
Myers	MYERG75,76									●	●		●					
NSSC PATHWAY	PATH76																	
Poole	DENNJ70																	
Richards	RICHP74											●						
Ruby	RUBYR68																	
SADPR-85	SUPP73			●	●													
SAMSO	SAMS75	●																
Schonfelder	SCHOJ76																	
Thayer	THAYT76																	
USA [illegible]SRAD	USAR75		●						●	●	●							
[illegible]	WHIPL75																	
Wulf	WULFW73										●							
Yourdon	YOURE75	●																
[illegible]	[illegible]									●	●	●		●	●	●	●	●

Quality Models: Boehm



Boehm, B. W., Brown, J. R., Kaspar, J. R., Lipow, M. L. & MacCleod, G., 1978. Characteristics of Software Quality. New York: American Elsevier.

System Quality Attributes

- Each have their own community (taxonomy, vocabulary, etc…)

- Definitions are not operational
- Fuzzy borders: is *system failure* an aspect of security, availability, or usability?

Quality attribute scenarios (spec of NFR)

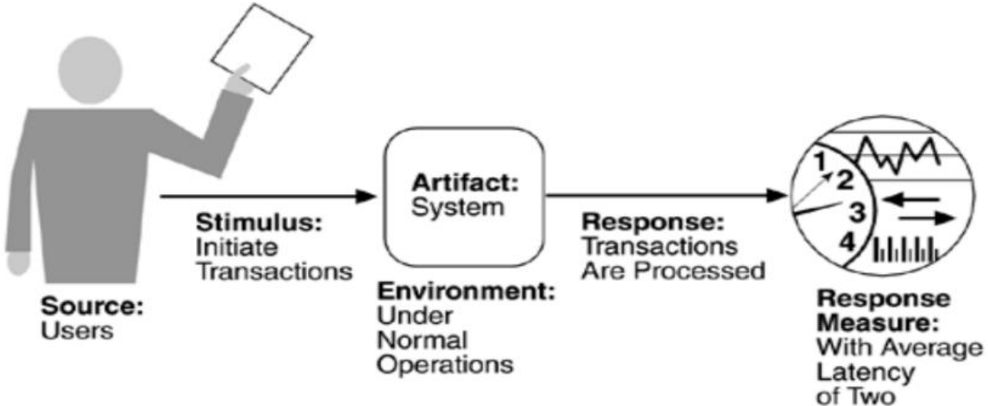
scenario-based + blackbox perspective

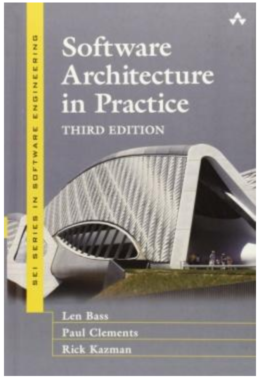
Quality attribute scenario elicitation

Handbook provides checklists

- Primary: *Availability*, *Interoperability*, *Modifiability*, *Performance*, *Security*, *Testability*, *Usability*
- Secondary: *Variability*, *Portability*, *Development distributability*, *Scalability/elasticity*, *Deployability*, *Mobility*, and *Monitorability*

Quality-attribute-specific tables to support creating system-specific scenarios



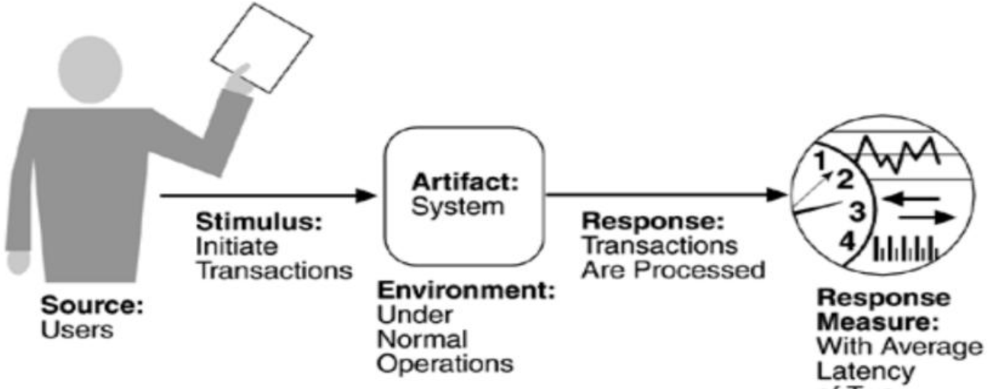


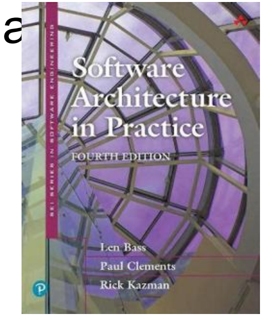
Quality attribute scenario elicitation

Handbook provides checklists

- Primary: *Availability*, ***Deployability***, ***Energy Efficiency***, ***Integrability***, *Modifiability*, *Performance*, ***Safety***, *Security*, *Testability*, *Usability*
- Secondary: ***Buildability***, ***Conceptual integrity***, ***Marketability***, *Development distributability*

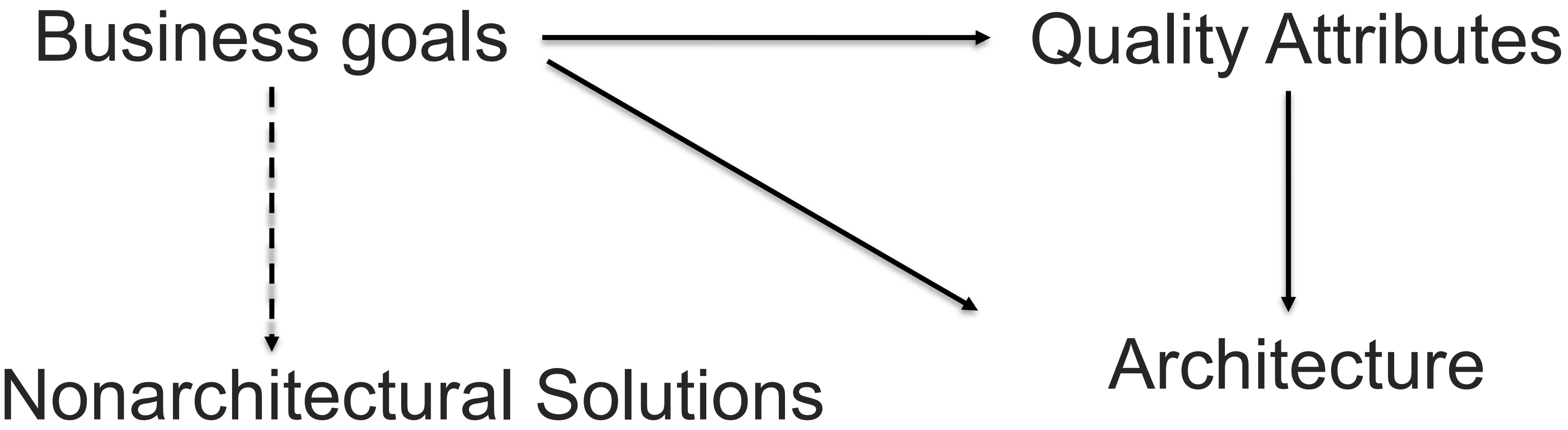
Quality-attribute-specific tables to support creating system-specific scena





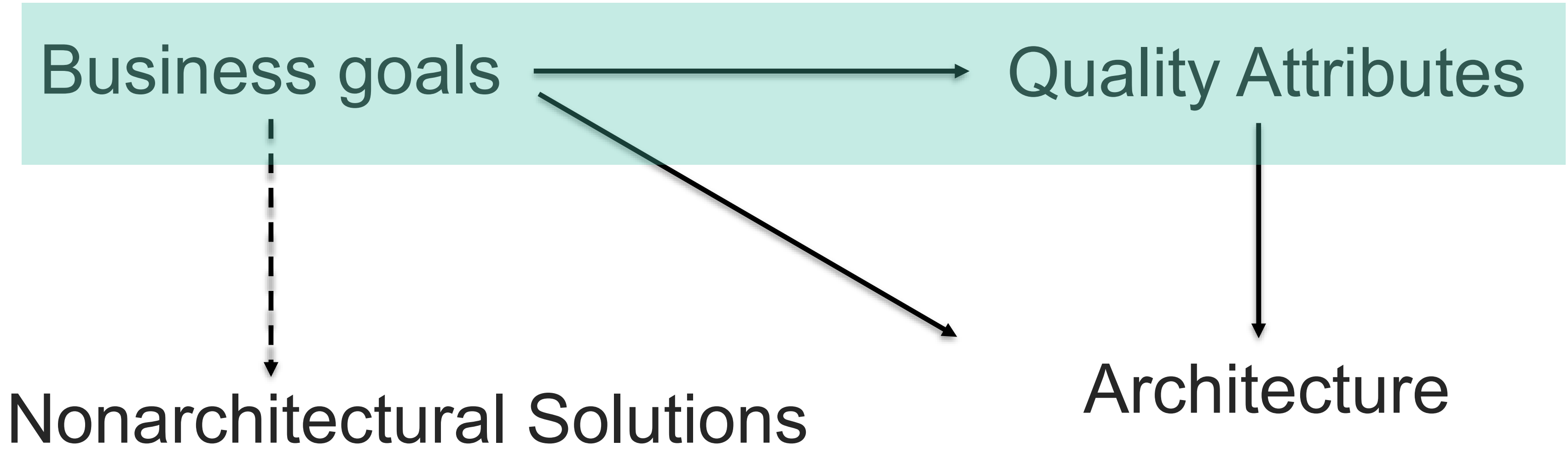
What makes requirements ***architecturally significant***?

Business goals



Software Architecture in Practice, ed 3; Chapter 3, Figure 3.2

Business goals



Software Architecture in Practice, ed 3; Chapter 3, Figure 3.2

Business goals

Gathering ASRs by understanding the business goals

- **Categories**
  - Contributing to the growth and continuity of the organization
  - Meeting financial objectives
  - Meeting personal objectives
  - Meeting responsibility to employees
  - Meeting responsibility to society
  - Meeting responsibility to state
  - Meeting responsibility to shareholders
  - Managing market position
  - Improving business processes
  - Managing the quality and reputation of products
  - Managing change in environmental factors

Software Architecture in Practice, ed 3; Chapter 16, Table 16.2

- ***Template*** (PALM):
  - Goal-source
  - Goal-subject
  - Goal-object
  - Environment
  - Goal
  - Goal-measure
  - Pedigree and value

Software Architecture in Practice, ed 3; Chapter 16

In this course:
good understanding of business goals is essential but they are documented informally

Architecturally significant requirements (ASRs)

- Criteria:
  1. **Architectural impact**: including the ASR will result in a very different architecture than if it were not included
  2. **Business or mission value**: importance to stakeholders
- Using a single list can help to evaluate each potential ASR against these criteria **and prioritize**

Architecturally significant requirements (ASRs)

What do you think?

- Color of the *add to shopping basket* button?

Architecturally significant requirements (ASRs)

What do you think?

- Color of the *add to shopping basket* button?
- Database performance?

Architecturally significant requirements (ASRs)

What do you think?

- Color of the *add to shopping basket* button?
- Database performance?
- Android system-level dark mode?

Architecturally significant requirements (ASRs)

What do you think?

- Color of the *add to shopping basket* button?
- Database performance?
- Android system-level dark mode?
- Authentication mechanism?

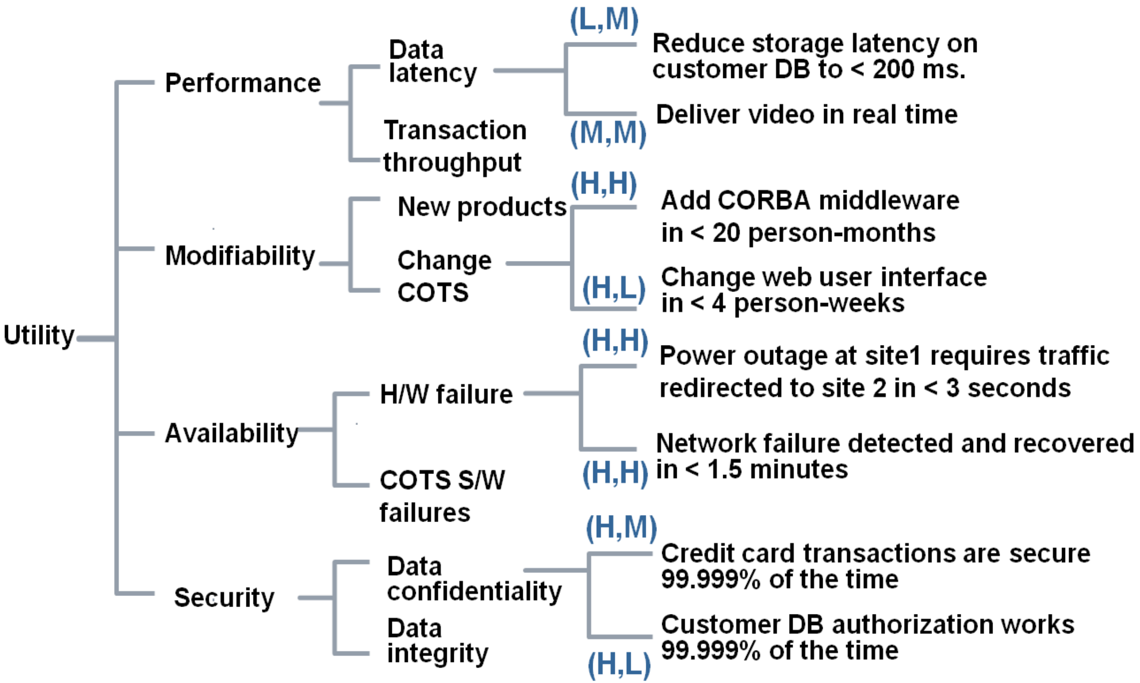
Architecturally significant requirements (ASRs)

What do you think?

- Color of the *add to shopping basket* button?
- Database performance?
- Android system-level dark mode?
- Authentication mechanism?
- Remote updating of car firmware?

Utility tree

- Four levels:
  1. Utility: expression of the overall *goodness* of a system
  2. Quality attribute
  3. Quality attribute refinement
  4. ASR incl. prio at the basis of <impact, value>



Recap

1. **Requirements engineering** is a separate discipline
2. **ASRs refer to software quality and are determined** by
    1. **business value,** and
    2. **architectural impact**

    Overview in **a Utility Tree**

    - Explicit motivation/rationale
3. **Documentation in Quality Attribute Scenarios**
    - Concrete+detailed+Measurable

Part 1 of the assignment

Takeaways

Take away #1

First positioning of
'**software architecture**':

the **bridge between requirements and design**

Take away #2

We do not perform requirements analysis

“for the sake of requirements analysis”

we pragmatically explore the key requirements of the system

Breadth first/brainstorm – ASRs
In-depth refinement of the “ones that matter most” - QASs

Take away #2

We do not perform requirements analysis

“for the sake of requirements analysis”

we pragmatically explore the architecturally significant requirements of the system

Breadth first/brainstorm – ASRs
In-depth refinement of the highest ranked NF requirements - QASs

Business value
Architectural impact

Software Architecture 2026
Lecture 1, part 3/3

Software engineering process and the role of requirements

Wouter Joosen

Laurens Sion

Dimitri Van Landuyt